Akademia Górniczo-Hutnicza w Krakowie			
Faculty: UBPO	Academic Year 2025/2026	2026	Fields of science:
Student(s) name:	**Simon GANIER-LOMBARD && Daniil STEPANOV**		
Course:	**Real Time Operating Systems in Control Applications**		
Exercise nr:	**RTOS_2**		
Exercise subject:	Real time IPC send and replies		
Lecturer:	dr inż. Grzegorz Hayduk		
Date: 19.03.2026			Grade:

1 Exercise purpose

The exercise is about communication between processes in QNX. It focuses on sending a message, receiving it, and replying. The goal is to understand how processes exchange data.

2 Assumptions / Theory

Processes communicate using three main functions: send, receive, and reply. When a process sends a message, it waits until it gets a reply. The receiver reads the message and sends a response back. This communication is synchronous and ensures data is correctly exchanged.

***

3 Description of implementation

Client.C

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include "common.h"
#include <unistd.h>

int main(int argc, char* argv[]) {
    my_data_t msg;
    int server_coid;

    if ((server_coid = name_open(ATTACH_POINT, 0)) == -1) {
        return EXIT_FAILURE;
    }
    int pid = getpid();
    // we display the process id after successful connection
    printf("name_open succeeded, my pid=%d\n", pid);

    // we initialize the message header
    msg.hdr.type    = 0x00;
    msg.hdr.subtype = 0x00;

    while (1) {
        // we ask the user to enter a word
        printf("Enter a word: ");

        // we read the user input from keyboard
        if (fgets(msg.data, 256, stdin) == NULL) {
            // we stop if input fails
            break;
        }
        // we get the the length of the input
        size_t len = strlen(msg.data);

        // we remove newline character if present
        if (len > 0 && msg.data[len - 1] == '\n') {
            msg.data[--len] = '\0';
        }

        // we check if the user typed "exit" in msg that is sending
        int is_exit = (strcmp(msg.data, "exit") == 0);
        printf("- calling MsgSend..\n");

        // we send the message to the server and wait for reply.
        int status = MsgSend(server_coid, &msg, sizeof(msg), &msg, sizeof(msg));

        // we stop if sending failed /if no the loop continue
        if (status == -1) {
            break;
        }
        // we print the reply received from the server
        printf("- msgReply.word = %s, status=%d\n", msg.data, status);

        // we exit the loop if the user typed "exit" / the loop continue otherwise
        if (is_exit) {
            printf("Client exit.\n");
            break;
        }
    }
```

```
    // we close the connection to the server
    name_close(server_coid);
    return EXIT_SUCCESS;
}
```

Server.C

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include "common.h"
#include <unistd.h>

int main(int argc, char *argv[]) {
    name_attach_t *attach;
    my_data_t     msg;
    int           rcvid;

    printf("Server application is starting...\n");

    if ((attach = name_attach(NULL, ATTACH_POINT, 0)) == NULL) {
        return EXIT_FAILURE;
    }

    int pid = getpid();

    printf("- name_attach succeeded, my pid=%d\n", pid);

    while (1) {
        // we wait for a message from a client
        printf("MsgReceive is waiting for a message...\n");
        rcvid = MsgReceive(attach->chid, &msg, sizeof(msg), NULL);

        // we stop if receive failed
        if (rcvid == -1) {
            break;
        }
        // we handle system pulses (not normal messages)
        if (rcvid == 0) { /* Pulse received */
            switch (msg.hdr.code) {
            case _PULSE_CODE_DISCONNECT:
                // we detach the connection when client disconnects
                ConnectDetach(msg.hdr.scoid);
                break;
            case _PULSE_CODE_UNBLOCK:
                break;
            default:
                break;
            }
            continue;
        }
        // handle special connect messages from sys
        if (msg.hdr.type == _IO_CONNECT) {
            MsgReply(rcvid, EOK, NULL, 0);
            continue;
        }
        // we reject unsupported io messages
        if (msg.hdr.type > _IO_BASE && msg.hdr.type <= _IO_MAX) {
            MsgError(rcvid, ENOSYS);
            continue;
        }
```

```
        // we check if client wants to exit and get the length of received word
        int is_exit = (strcmp(msg.data, "exit") == 0);
        int len     = (int)strlen(msg.data);
        char reversed[256];
        //  reverse the word character by character
        int i;
        for (i = 0; i < len; i++) {
            reversed[i] = msg.data[len - 1 - i];
        }
        reversed[len] = '\0';

        // build the response with length and reversed word
        snprintf(msg.data, 256, "%d:%s", len, reversed);
        printf("- received word, replying with %s\n", msg.data);
        MsgReply(rcvid, len, &msg, sizeof(msg));

        // we exit loop if client sent "exit"
        if (is_exit) {
            break;
        }
    }

    name_detach(attach, 0);
    printf("Server exit.\n");

    return EXIT_SUCCESS;
}
```

Conclusions

We appreciate to re-use C to code some feature, that as being a longtime we didn't use C like this in our previous school. We also learn more about the IPC communication inside QNX and how to make asynchronous connection between two process.